

Wydział Informatyki Politechniki Białostockiej

Systemy wbudowane

Tytuł: Automaty skończone

Wykonujący: Łukasz Modzelewski, Mariusz Rostkowski

PS 1

Zespół 4

Praca laboratoryjna 9

Studia dzienne

Informatyka

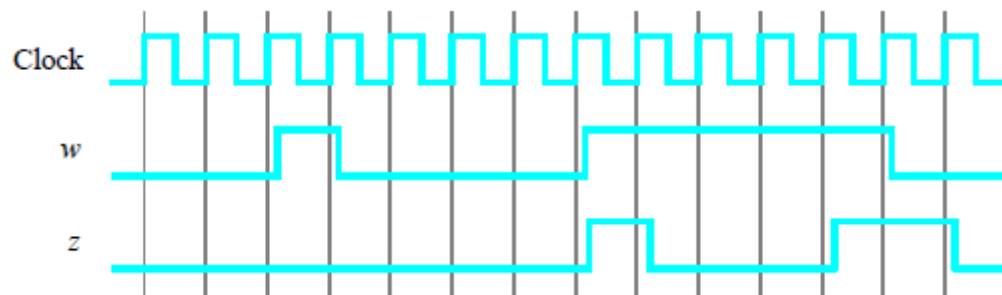
Semestr VI

Prowadzący: prof. dr hab. inż. Valery Salauyou

Data: 05.05.2022

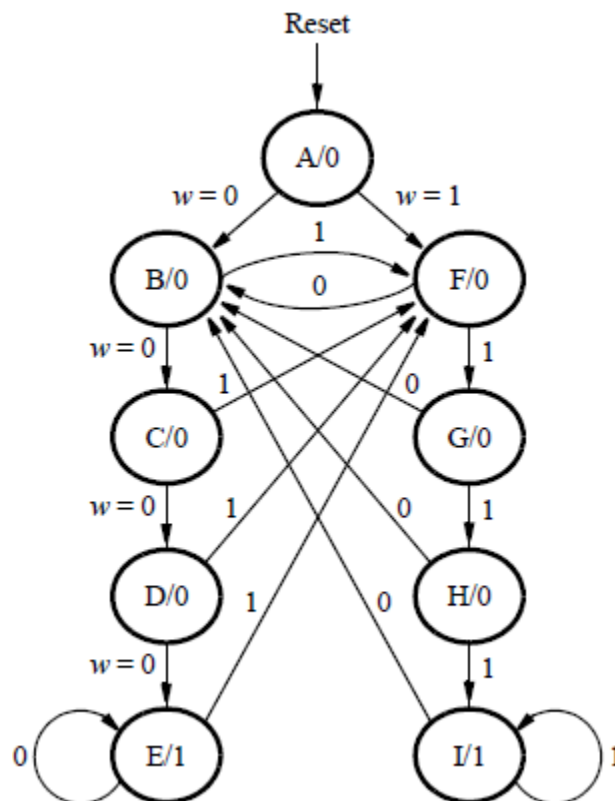
Zadanie 1

Zaimplementuj skończoną maszynę stanów, która na wejściu w rozpoznaje sekwencje czterech jedynek lub czterech zer. Za każdym razem, gdy taka sekwencja zostanie znaleziona, wynikiem jest $z = 1$, w przeciwnym przypadku $z = 0$. Dozwolone są sekwencje nakładające się, czyli jeśli $z = 1$ po czterech jedynekach (zerach), to z będzie 1 i po pięciu, sześciu, itd. jedynekach (zerach). Przykład relacji między w i z pokazano na rys. 1.



Rys. 1. Wymagana synchronizacja dla wyjścia z

Diagram stanu (graf automatu) pokazano na rys. 2.



Rys. 2. Diagram stanów dla FSM

Utwórz projekt FSM_one_hot skończonej maszyny stanów w przypadku kodowania unarnego (one-hot).

Tabela 1. Kody one-hot dla FSM

Name	State Code
	$y_8y_7y_6y_5y_4y_3y_2y_1y_0$
A	000000001
B	000000010
C	000000100
D	000001000
E	000010000
F	000100000
G	001000000
H	010000000
I	100000000

Używaj tylko prostych operatorów przypisania do określania wartości na wejściach przerzutników (nie możesz używać instrukcji case i if). Przetestuj projekt za pomocą następującego przypisania pinów:

Wyprowadzenia	Piny
w	SW1
clk	KEY0
aclr	SW0
z	LEDR9
y[8:0]	LEDR[8:0]

gdzie **y[8:0]** – Wyjścia przerzutników definiujące kody stanu automatu skończonego.

Kod w Verilog

Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty

```
module FSM_one_hot_on_board(
    input [1:0] SW,
    input [0:0] KEY,
    output [9:0] LEDR);

    FSM_one_hot fsm(SW[1], KEY[0], ~SW[0], LEDR[9], LEDR[8:0]);

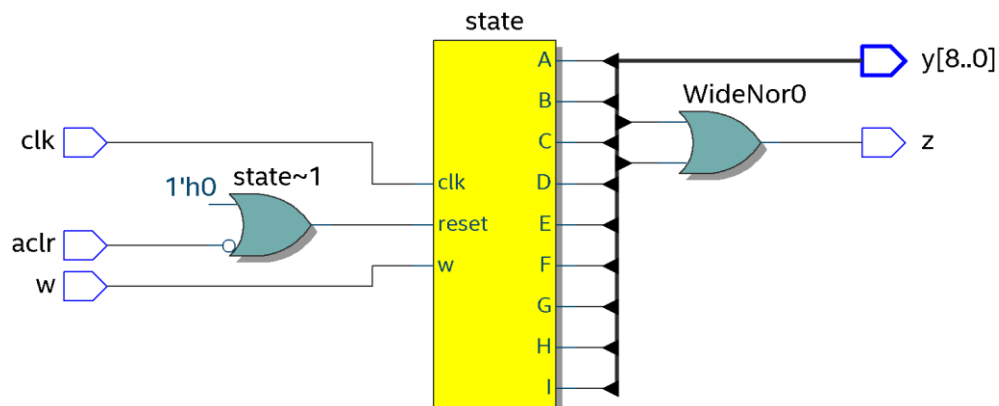
endmodule
```

Moduł automatu skończonego z kodowaniem one-hot

```
module FSM_one_hot(  
    input w, clk, aclr,  
    output reg z,  
    output [8:0] y);  
  
    localparam [8:0]  
        A = 9'b000000001,  
        B = 9'b000000010,  
        C = 9'b000000100,  
        D = 9'b000001000,  
        E = 9'b000010000,  
        F = 9'b000100000,  
        G = 9'b001000000,  
        H = 9'b010000000,  
        I = 9'b100000000;  
    reg [8:0] state = A, next;  
    assign y = state;  
  
    always @ (posedge clk, negedge aclr)  
        if (~aclr) state <= A;  
        else state <= next;  
  
    always @ (*)  
        case (state)  
            A: if (w) next = F; else next = B;  
            B: if (w) next = F; else next = C;  
            C: if (w) next = F; else next = D;  
            D: if (w) next = F; else next = E;  
            E: if (w) next = F; else next = E;  
            F: if (w) next = G; else next = B;  
            G: if (w) next = H; else next = B;  
            H: if (w) next = I; else next = B;  
            I: if (w) next = I; else next = B;  
            default: next = A;  
        endcase  
  
    always @ (*)  
        case (state)  
            E, I: z = 1'b1;  
            default: z = 1'b0;  
        endcase  
  
endmodule
```

Wyniki syntezy

RTL Viewer



State Machine Viewer

Graf stanów

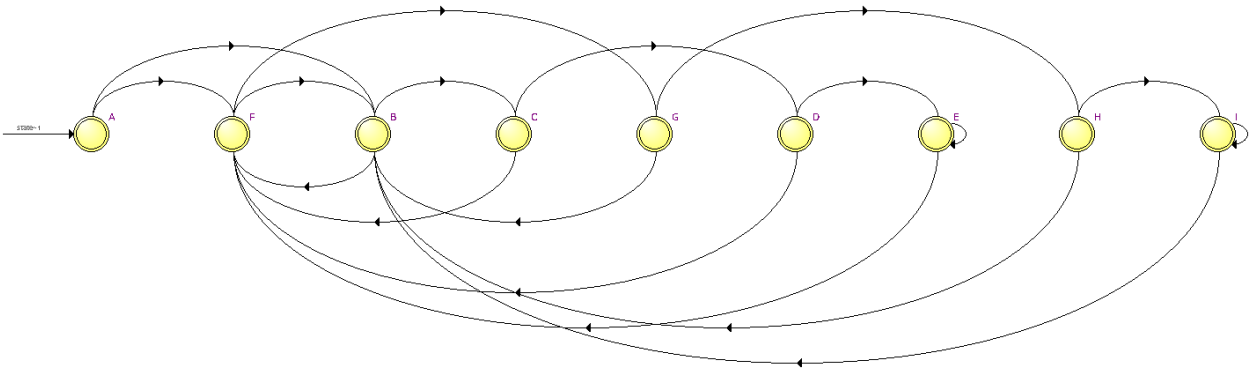


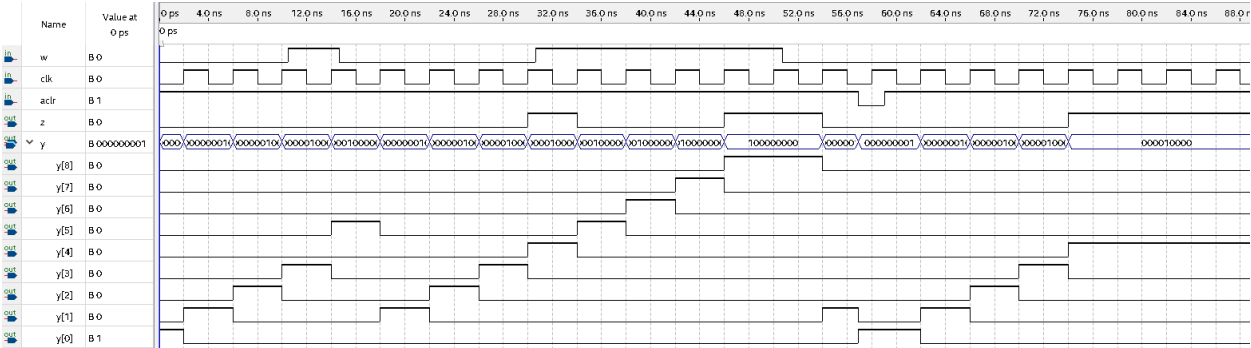
Tabela przejść

	Source State	Destination State	Condition
1	A	B	{!w}
2	A	F	{w}
3	B	F	{w}
4	B	C	{!w}
5	C	D	{!w}
6	C	F	{w}
7	D	F	{w}
8	D	E	{!w}
9	E	F	{w}
10	E	E	{!w}
11	F	B	{!w}
12	F	G	{w}
13	G	B	{!w}
14	G	H	{w}
15	H	B	{!w}
16	H	I	{w}
17	I	B	{!w}
18	I	I	{w}

Tabela kodowania

Name	state~12	state~11	state~10	state~9	state~8	state~7	state~6	state~5	state~4
1 A	0	0	0	0	0	0	0	0	0
2 B	0	0	0	0	0	0	0	1	1
3 C	0	0	0	0	0	0	1	0	1
4 D	0	0	0	0	0	1	0	0	1
5 E	0	0	0	0	1	0	0	0	1
6 F	0	0	0	1	0	0	0	0	1
7 G	0	0	1	0	0	0	0	0	1
8 H	0	1	0	0	0	0	0	0	1
9 I	1	0	0	0	0	0	0	0	1

Wyniki symulacji funkcjonalnej



Zadanie 2

Wykonaj skończoną maszynę stanów z rys. 2 (projekt FSM_user_coding) używając instrukcji case i if (jeśli to konieczne). Przypisz następujące kody do stanów wewnętrznych:

Tabela 3. Kody binarne dla FSM

Name	State Code
	$y_3y_2y_1y_0$
A	0000
B	0001
C	0010
D	0011
E	0100
F	0101
G	0110
H	0111
I	1000

W Quartus ustaw parametr **Assignment > Settings > Compiler Settings > Advanced Settings (Synthesis) > State Machine Processing > User Encoded**, aby Quartus użył kodów stanu określonych przez użytkownika w kodzie projektu.

Kod w Verilog

Moduł automatu skończonego z kodowaniem z tabeli 3.

```
module FSM_user_coding(
    input w, clk, aclr,
    output reg z,
    output [3:0] y);

    reg [3:0] state, next;
    assign y = state;
    localparam [3:0]
        A = 4'b0000, // 0
        B = 4'b0001, // 1
        C = 4'b0010, // 2
        D = 4'b0011, // 3
        E = 4'b0100, // 4
        F = 4'b0101, // 5
        G = 4'b0110, // 6
        H = 4'b0111, // 7
        I = 4'b1000; // 8

    always @ (posedge clk, negedge aclr)
        if (~aclr) state <= A;
        else state <= next;
```

```

always @ (*) // opis funkcji przejść
case (state)
  A: if (w) next = F; else next = B;
  B: if (w) next = F; else next = C;
  C: if (w) next = F; else next = D;
  D: if (w) next = F; else next = E;
  E: if (w) next = F; else next = E;
  F: if (w) next = G; else next = B;
  G: if (w) next = H; else next = B;
  H: if (w) next = I; else next = B;
  I: if (w) next = I; else next = B;
  default: next = A;
endcase

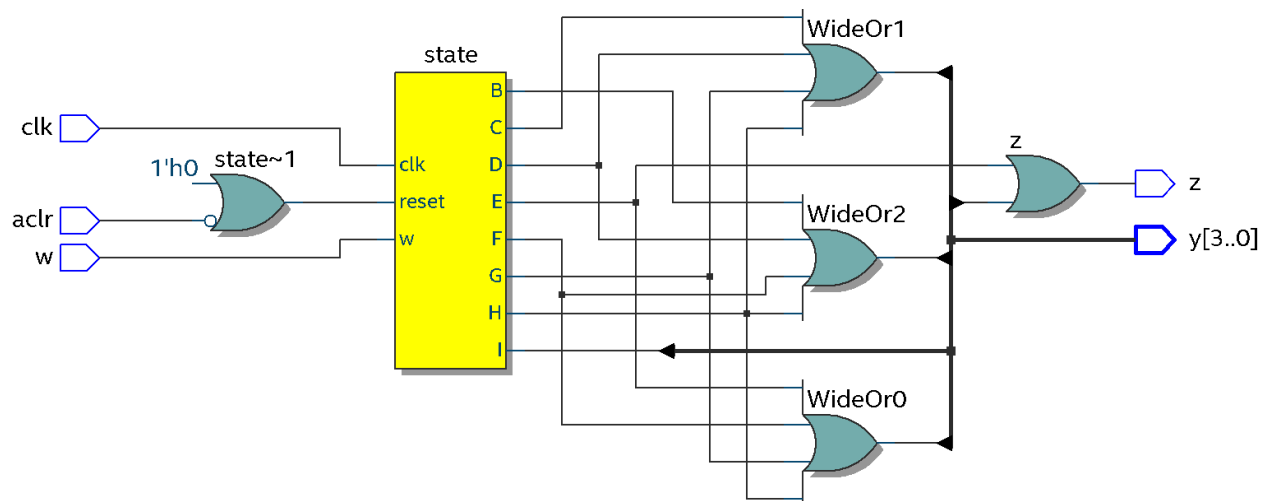
always @ (*) // opis funkcji wyjść
case (state)
  A: z = 1'b0;
  B: z = 1'b0;
  C: z = 1'b0;
  D: z = 1'b0;
  E: z = 1'b1;
  F: z = 1'b0;
  G: z = 1'b0;
  H: z = 1'b0;
  I: z = 1'b1;
  default: z = 1'b0;
endcase

endmodule

```

Wyniki syntezy

RTL Viewer



State Machine Viewer

Graf stanów

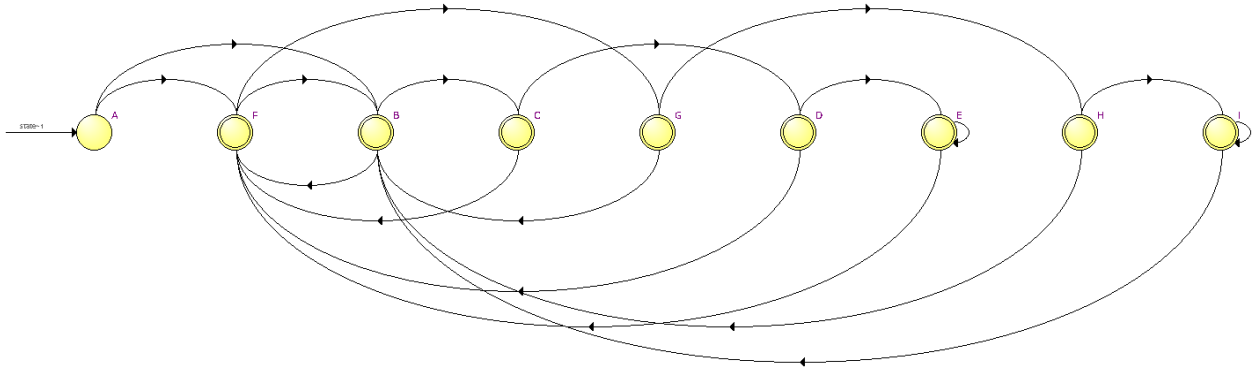


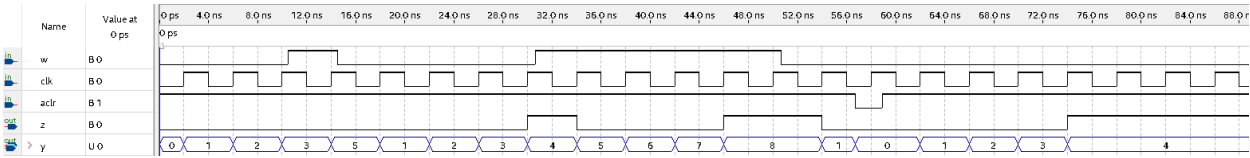
Tabela przejść

	Source State	Destination State	Condition
1	A	B	{!w}
2	A	F	{w}
3	B	C	{!w}
4	B	F	{w}
5	C	D	{!w}
6	C	F	{w}
7	D	E	{!w}
8	D	F	{w}
9	E	E	{!w}
10	E	F	{w}
11	F	B	{!w}
12	F	G	{w}
13	G	B	{!w}
14	G	H	{w}
15	H	B	{!w}
16	H	I	{w}
17	I	B	{!w}
18	I	I	{w}

Tabela kodowania

	Name	state~7	state~6	state~5	state~4
1	A	0	0	0	0
2	B	0	0	0	1
3	C	0	0	1	0
4	D	0	0	1	1
5	E	0	1	0	0
6	F	0	1	0	1
7	G	0	1	1	0
8	H	0	1	1	1
9	I	1	0	0	0

Wyniki symulacji funkcjonalnej



Zadanie 3

Utwórz 3 implementacje automatu skończonego z zad. 2: z trzema procesami, dwoma procesami i jednym procesem. Przeanalizuj wszystkie trzy implementacje. Porównaj je względem kosztu realizacji i szybkości (dla tego samego kodowania). Porównaj także wyniki syntezy automatów skończonych: **Tools > Netlist Viewers > State Machine Viewer**.

Kod w Verilog

Moduł automatu skończonego z trzema procesami

```
module FSM_3_always(  
    input w, clk, aclr,  
    output reg z,  
    output [3:0] y);  
  
    reg [3:0] state, next;  
    assign y = state;  
    localparam [3:0]  
        A = 4'b0000, // 0  
        B = 4'b0001, // 1  
        C = 4'b0010, // 2  
        D = 4'b0011, // 3  
        E = 4'b0100, // 4  
        F = 4'b0101, // 5  
        G = 4'b0110, // 6  
        H = 4'b0111, // 7  
        I = 4'b1000; // 8  
  
    always @ (posedge clk, negedge aclr)  
        if (~aclr) state <= A;  
        else state <= next;  
  
    always @ (*)  
        case (state)  
            A: if (w) next = F; else next = B;  
            B: if (w) next = F; else next = C;  
            C: if (w) next = F; else next = D;  
            D: if (w) next = F; else next = E;  
            E: if (w) next = F; else next = E;  
            F: if (w) next = G; else next = B;  
            G: if (w) next = H; else next = B;  
            H: if (w) next = I; else next = B;  
            I: if (w) next = I; else next = B;  
            default: next = A;  
        endcase  
  
    always @ (*)  
        case (state)  
            A: z = 1'b0;  
            B: z = 1'b0;  
            C: z = 1'b0;  
            D: z = 1'b0;  
            E: z = 1'b1;  
            F: z = 1'b0;  
            G: z = 1'b0;  
            H: z = 1'b0;  
            I: z = 1'b1;  
            default: z = 1'b0;  
        endcase  
  
endmodule
```

Moduł automatu skończonego z dwoma procesami. Łączymy wyznaczenie wyjścia i następnego stanu w jednym procesie. Zmiana stanu na następny pozostaje w oddzielnym procesie.

```
module FSM_2_always(  
    input w, clk, aclr,  
    output reg z,  
    output [3:0] y);  
  
    reg [3:0] state, next;  
    assign y = state;  
    localparam [3:0]  
        A = 4'b0000, // 0  
        B = 4'b0001, // 1  
        C = 4'b0010, // 2  
        D = 4'b0011, // 3  
        E = 4'b0100, // 4  
        F = 4'b0101, // 5  
        G = 4'b0110, // 6  
        H = 4'b0111, // 7  
        I = 4'b1000; // 8  
  
    always @ (posedge clk, negedge aclr)  
        if (~aclr) state <= A;  
        else state <= next;  
  
    always @ (*)  
        case (state)  
            A: begin z = 1'b0; if (w) next = F; else next = B; end  
            B: begin z = 1'b0; if (w) next = F; else next = C; end  
            C: begin z = 1'b0; if (w) next = F; else next = D; end  
            D: begin z = 1'b0; if (w) next = F; else next = E; end  
            E: begin z = 1'b1; if (w) next = F; else next = E; end  
            F: begin z = 1'b0; if (w) next = G; else next = B; end  
            G: begin z = 1'b0; if (w) next = H; else next = B; end  
            H: begin z = 1'b0; if (w) next = I; else next = B; end  
            I: begin z = 1'b1; if (w) next = I; else next = B; end  
            default: begin z = 1'b0; next = A; end  
        endcase  
  
endmodule
```

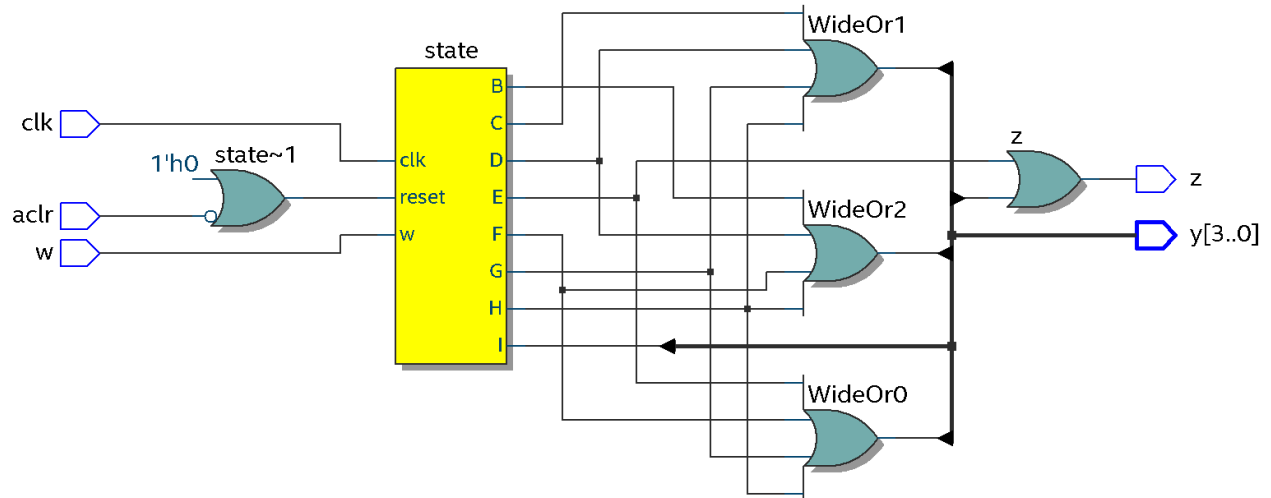
Moduł automatu skończonego z jednym procesem. Zmiana stanu znajduje się w jedynym procesie i nie używamy zmiennej opisującej stan następny. Wyjście opisujemy za pomocą operatorów warunkowych.

```
module FSM_1_always(  
    input w, clk, aclr,  
    output z,  
    output [3:0] y);  
  
    reg [3:0] state;  
    assign y = state;  
    localparam [3:0]  
        A = 4'b0000, // 0  
        B = 4'b0001, // 1  
        C = 4'b0010, // 2  
        D = 4'b0011, // 3  
        E = 4'b0100, // 4  
        F = 4'b0101, // 5  
        G = 4'b0110, // 6  
        H = 4'b0111, // 7  
        I = 4'b1000; // 8  
  
    always @ (posedge clk, negedge aclr)  
        if (~aclr) state <= A;  
        else case (state)  
            A: if (w) state = F; else state = B;  
            B: if (w) state = F; else state = C;  
            C: if (w) state = F; else state = D;  
            D: if (w) state = F; else state = E;  
            E: if (w) state = F; else state = E;  
            F: if (w) state = G; else state = B;  
            G: if (w) state = H; else state = B;  
            H: if (w) state = I; else state = B;  
            I: if (w) state = I; else state = B;  
            default: state = A;  
        endcase  
  
    assign z = ((state == A) ? 1'b0 : (  
        (state == B) ? 1'b0 : (  
        (state == C) ? 1'b0 : (  
        (state == D) ? 1'b0 : (  
        (state == E) ? 1'b1 : (  
        (state == F) ? 1'b0 : (  
        (state == G) ? 1'b0 : (  
        (state == H) ? 1'b0 : (  
        (state == I) ? 1'b1 : 1'b0)))))))));  
  
endmodule
```

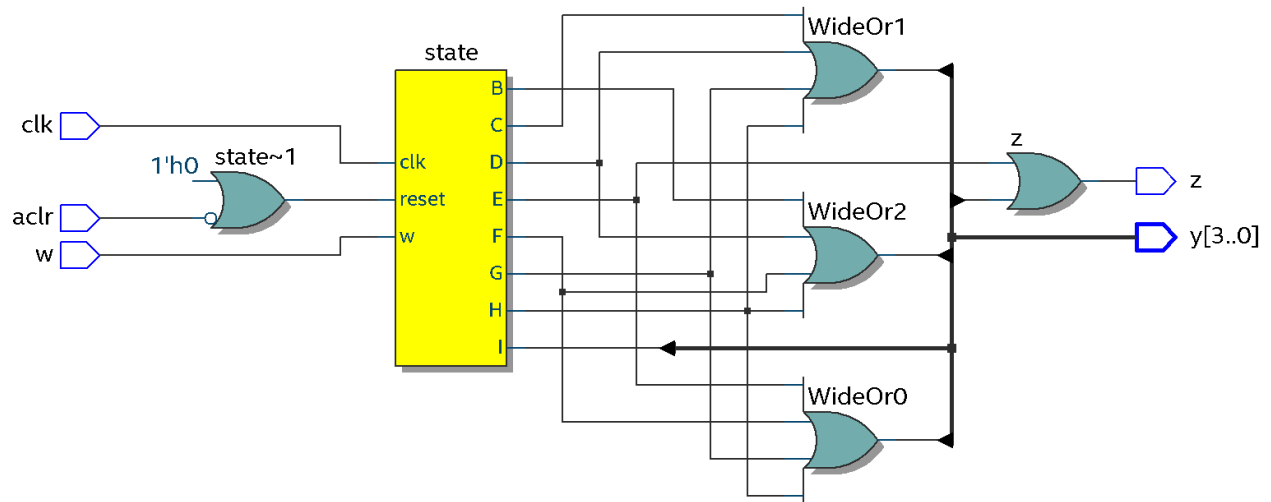
Wyniki syntezy

RTL Viewer

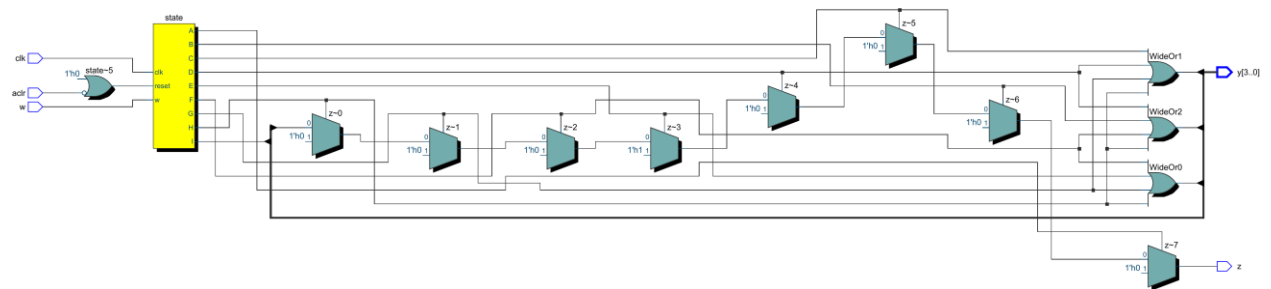
Automat skończony z trzema procesami



Automat skończony z dwoma procesami



Automat skończony z jednym procesem



State Machine Viewer

Automat skończony z trzema procesami

Graf stanów

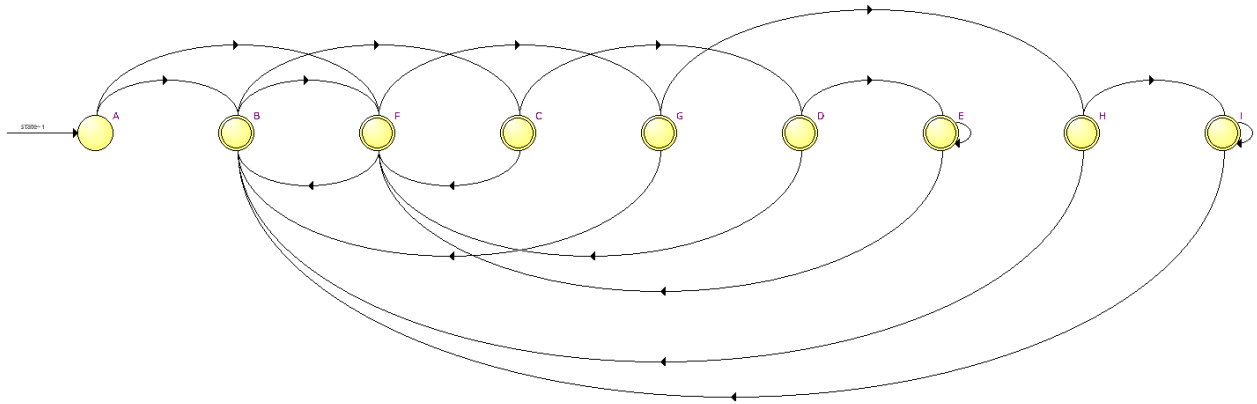


Tabela przejść

	Source State	Destination State	Condition
1	A	B	{!w}
2	A	F	{w}
3	B	C	{!w}
4	B	F	{w}
5	C	D	{!w}
6	C	F	{w}
7	D	E	{!w}
8	D	F	{w}
9	E	E	{!w}
10	E	F	{w}
11	F	B	{!w}
12	F	G	{w}
13	G	B	{!w}
14	G	H	{w}
15	H	B	{!w}
16	H	I	{w}
17	I	B	{!w}
18	I	I	{w}

Tabela kodowania

	Name	state~7	state~6	state~5	state~4
1	A	0	0	0	0
2	B	0	0	0	1
3	C	0	0	1	0
4	D	0	0	1	1
5	E	0	1	0	0
6	F	0	1	0	1
7	G	0	1	1	0
8	H	0	1	1	1
9	I	1	0	0	0

Automat skończony z dwoma procesami

Graf stanów

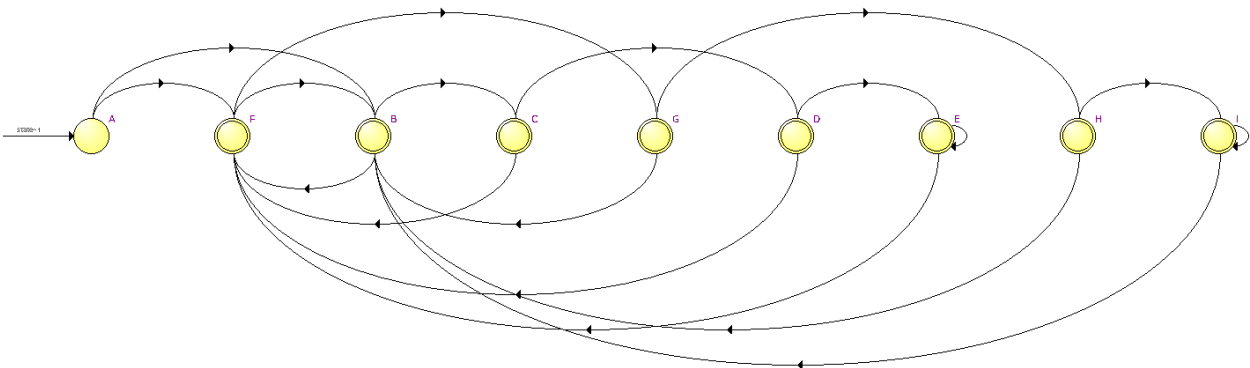


Tabela przejść

	Source State	Destination State	Condition
1	A	B	{!w}
2	A	F	{w}
3	B	F	{w}
4	B	C	{!w}
5	C	F	{w}
6	C	D	{!w}
7	D	F	{w}
8	D	E	{!w}
9	E	F	{w}
10	E	E	{!w}
11	F	G	{w}
12	F	B	{!w}
13	G	B	{!w}
14	G	H	{w}
15	H	B	{!w}
16	H	I	{w}
17	I	B	{!w}
18	I	I	{w}

Tabela kodowania

	Name	state~7	state~6	state~5	state~4
1	A	0	0	0	0
2	B	0	0	0	1
3	C	0	0	1	0
4	D	0	0	1	1
5	E	0	1	0	0
6	F	0	1	0	1
7	G	0	1	1	0
8	H	0	1	1	1
9	I	1	0	0	0

Automat skończony z jednym procesem

Graf stanów

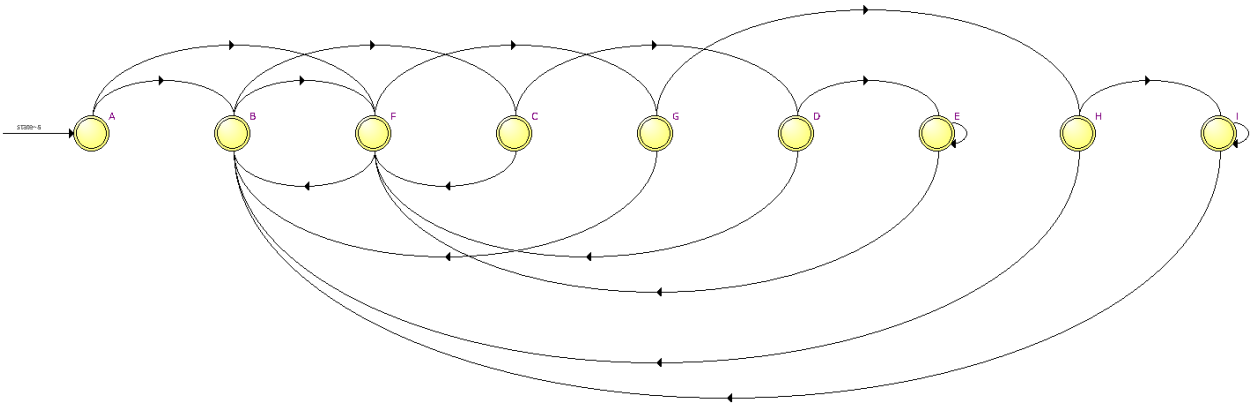


Tabela przejść

	Source State	Destination State	Condition
1	A	B	{!w}
2	A	F	{w}
3	B	C	{!w}
4	B	F	{w}
5	C	D	{!w}
6	C	F	{w}
7	D	E	{!w}
8	D	F	{w}
9	E	E	{!w}
10	E	F	{w}
11	F	B	{!w}
12	F	G	{w}
13	G	B	{!w}
14	G	H	{w}
15	H	B	{!w}
16	H	I	{w}
17	I	B	{!w}
18	I	I	{w}

Tabela kodowania

	Name	state~7	state~6	state~5	state~4
1	A	0	0	0	0
2	B	0	0	0	1
3	C	0	0	1	0
4	D	0	0	1	1
5	E	0	1	0	0
6	F	0	1	0	1
7	G	0	1	1	0
8	H	0	1	1	1
9	I	1	0	0	0

Koszt zasobów sprzętowych

Automat skończony z trzema procesami

Flow Status	Successful - Thu May 12 01:30:30 2022
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	FSM_3_always
Top-level Entity Name	FSM_3_always
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	3 / 32,070 (< 1 %)
Total registers	8
Total pins	8 / 457 (2 %)

Automat skończony z dwoma procesami

Flow Status	Successful - Thu May 12 01:30:29 2022
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	FSM_2_always
Top-level Entity Name	FSM_2_always
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	3 / 32,070 (< 1 %)
Total registers	8
Total pins	8 / 457 (2 %)

Automat skończony z jednym procesem

Flow Status	Successful - Thu May 12 01:30:29 2022
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	FSM_1_always
Top-level Entity Name	FSM_1_always
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	3 / 32,070 (< 1 %)
Total registers	8
Total pins	8 / 457 (2 %)

Maksymalna częstotliwość zegara

Automat skończony z trzema procesami

	Fmax	Restricted Fmax	Clock Name
1	248.94 MHz	248.94 MHz	clk

Automat skończony z dwoma procesami

	Fmax	Restricted Fmax	Clock Name
1	248.94 MHz	248.94 MHz	clk

Automat skończony z jednym procesem

	Fmax	Restricted Fmax	Clock Name
1	258.67 MHz	258.67 MHz	clk

Graf stanów oraz tabele przejść i kodowania wszystkich trzech wersji automatu skończonego są równe. W RTL Viewer wyniki syntezy automatu z trzema i dwoma procesami są równe. Automat z jednym procesem zawiera kilka multiplexerów, ponieważ wyjście „z” opisaaliśmy za pomocą zagnieżdżonych operatorów warunkowych.

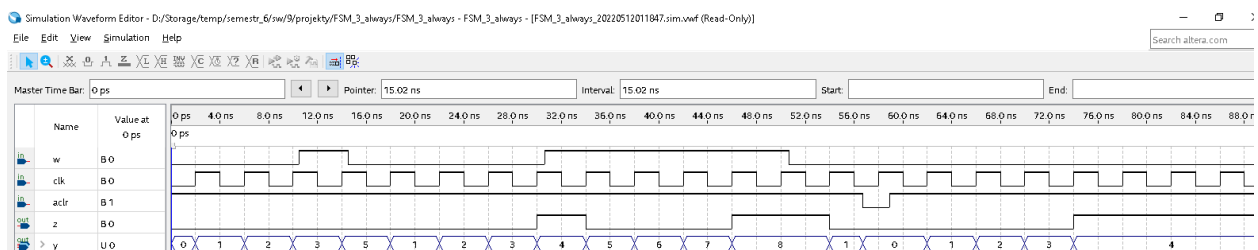
Z kosztów zasobów sprzętowych wynika, że wszystkie trzy wersje automatu używają 3 modułów logicznych (ALM) i 8 rejestrów, więc koszt ich realizacji jest taki sam.

Automaty z trzema i dwoma procesami są w stanie pracować z sygnałem zegarowym clk o maksymalnej częstotliwości 248.94 MHz. Natomiast automat z jednym procesem z 258.67 MHz, więc może być szybszy od wersji z trzema i dwoma procesami.

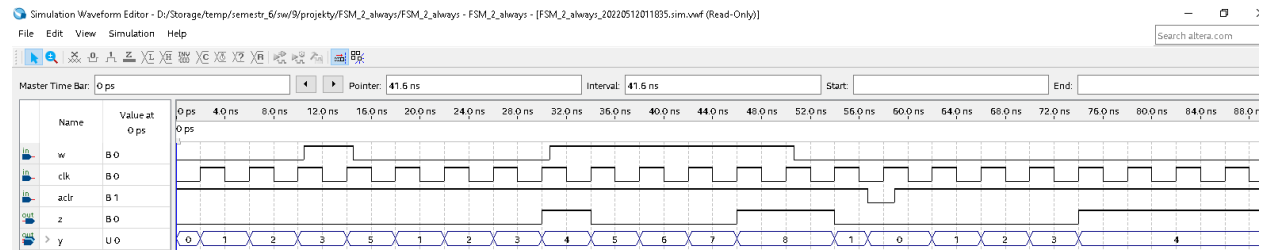
Automat z jednym procesem jest najlepszą wersją, ponieważ ma taki sam koszt realizacji jak 2 pozostałe wersje, ale może pracować z zegarem o większej od nich częstotliwości.

Wyniki symulacji funkcjonalnej

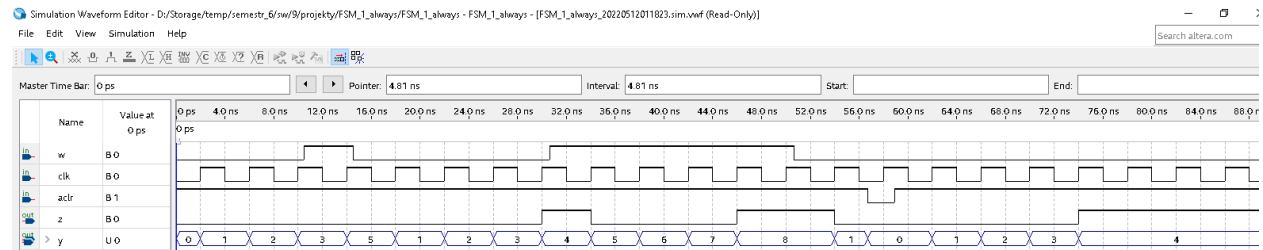
Automat skończony z trzema procesami



Automat skończony z dwoma procesami



Automat skończony z jednym procesem



Wszystkie wersje dają takie same wyniki symulacji funkcjonalnej.

Zadanie 4

Utwórz projekt `detector_with_shift_reg`, który implementuje detektor sekwencji na rejestrach przesuwanych. W tym celu należy utworzyć dwa 4-bitowe rejestry przesuwne, jeden do rozpoznawania sekwencji jedynek, a drugi do zer. Dołącz niezbędne wyrażenia logiczne do projektu, aby utworzyć wyjście `z`. Porównaj projekt z poprzednimi implementacjami pod względem kosztu realizacji i szybkości.

Kod w Verilog

Moduł detektora sekwencji na rejestrach przesuwanych. „Synthesis keep” używamy tylko do symulacji i usuwamy je przed kompilacją w celu określenia maksymalnej częstotliwości zegara.

```
module detector_with_shift_reg(
    input w, clk, aclr,
    output z);

    wire [3:0] sr0_state, sr1_state /* synthesis keep */;
    shift_register #(4, 4'b1111) sr0(clk, aclr, w, sr0_state);
    shift_register #(4, 4'b0000) sr1(clk, aclr, w, sr1_state);
    assign z = (~|sr0_state) | (&sr1_state);

endmodule
```

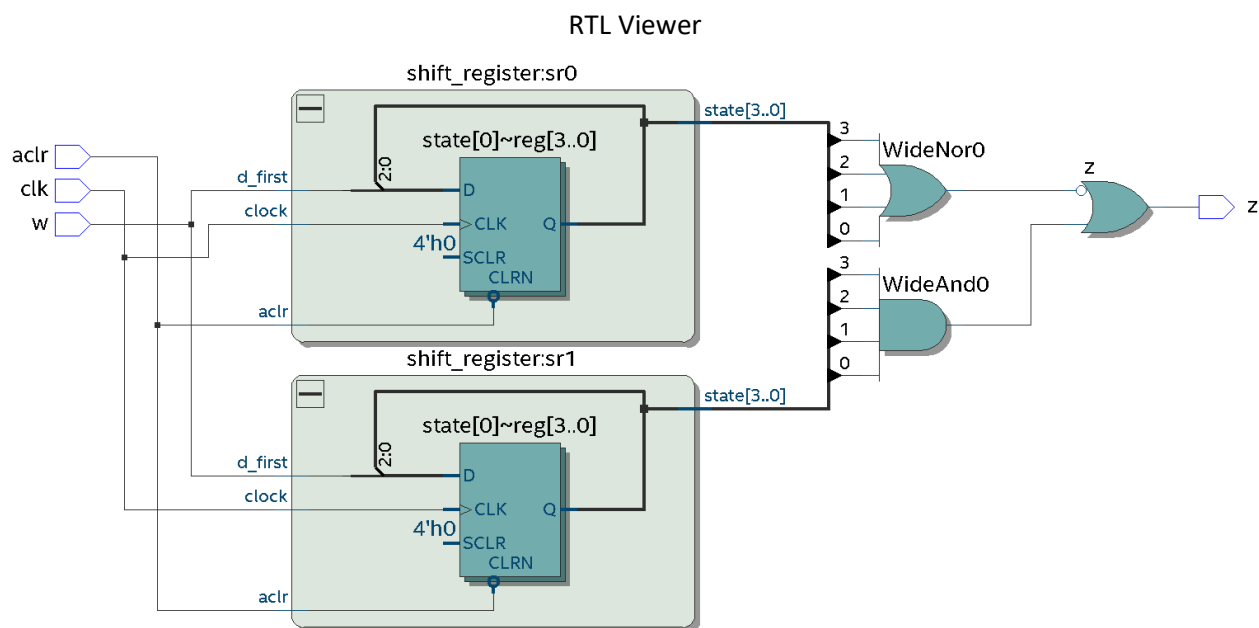
Moduł rejestru przesuwanego

```
module shift_register
    #(parameter N = 4, INITIAL_VALUE = 4'b0000)(
    input clock, aclr,
    input d_first,
    output reg [N-1:0] state);

    initial state = INITIAL_VALUE;
    always @ (posedge clock, negedge aclr)
        if (~aclr) state <= INITIAL_VALUE;
        else state <= {state[N-2:0], d_first};

endmodule
```

Wyniki syntezy



Koszt zasobów sprzętowych

Flow Status	Successful - Thu May 12 05:01:41 2022
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	detector_with_shift_reg
Top-level Entity Name	detector_with_shift_reg
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	3 / 32,070 (< 1 %)
Total registers	8
Total pins	4 / 457 (< 1 %)

Maksymalna częstotliwość zegara

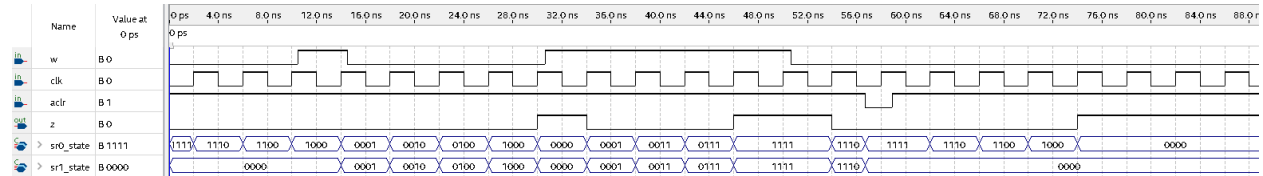
	Fmax	Restricted Fmax	Clock Name	Note
1	973.71 MHz	717.36 MHz	clk	limit due to minimum period restriction (tmin)

Z kosztów zasobów sprzętowych wynika, że układ używa 3 modułów logicznych (ALM) i 8 rejestrów, więc koszt jego realizacji jest taki sam jak każdej z 3 wersji (z trzema, dwoma i jednym procesem) automatu skończonego rozpoznającego sekwencje czterech jedynek lub zer.

Układ jest w stanie pracować z sygnałem zegarowym `clk` o maksymalnej częstotliwości 717.36 MHz, więc może być szybszy od każdej z 3 wersji automatu skończonego.

Wyniki symulacji funkcjonalnej

Rejestr `sr0_state` ma początkową zawartość `1111` i po wprowadzeniu zer w 4 kolejnych taktach zegara `clk` wyjście `z` przyjmuje wartość `1`. Analogicznie rejestr `sr1_state` ma początkową wartość `0000` i po wprowadzeniu jedynek w 4 kolejnych taktach zegara `clk` wyjście `z` przyjmuje wartość `1`.



Wnioski

Wszystkie zadania zostały zrealizowane.